

CodeCaster: A Vibe-Coded Platform for AI-Assisted Social Science Data Analysis

Yibarek Tadesse
William & Mary

May 5, 2026

Abstract

Social science students and researchers frequently encounter a significant barrier to entry when conducting quantitative analysis: the steep learning curve of programming languages like Python and R. Standard analysis workflows require scrubbing data, handling null values, and mastering complex syntax, which detracts from time spent interpreting actual findings. This project delivers CodeCaster, a web-based, AI-driven data analysis platform that abstracts away the coding process. Users upload datasets and use natural language to request analytical workflows acting as project managers. The platform leverages a multi-model Gemini architecture to generate, validate, and securely execute Python or R scripts in a local Dockerized sandbox, natively returning statistical interpretations and visualizations. Built end-to-end via vibe coding, CodeCaster demonstrates the power of AI as a development partner. This report documents the market need, the technical architecture, and reflections on the vibe coding process.

1 Introduction

Throughout my undergraduate years at William & Mary, I observed a recurring hurdle in social science courses: students are frequently asked to draw meaningful analytical observations from large datasets, yet many lack a foundational background in Computer Science. This gap forces researchers to step out of their comfort zone, spending countless hours fighting syntax errors rather than analyzing voter behavior, economic trends, or social phenomena. While Large Language Models (LLMs) exist to help write code, they require prompt engineering savviness. Furthermore, passing raw, dense CSV data back and forth into a generalized chatbot is an arduous, error-prone process that lacks context awareness. This project addresses that gap by building CodeCaster, a dedicated interactive sandbox where users of any technical background can securely upload datasets and use natural language to direct a specialized AI agent. The platform generates the exact script needed, executes it locally, and interprets the results. I will document the specifics of CodeCaster and explaining my experience with “vibe coding” and how AI tools were utilized to build this project.

2 Market Analysis

2.1 Target Segment

CodeCaster targets college students, academic researchers, and industry analysts within the social sciences. The primary users are individuals who need to run summary statistics, data filtering, and complex multivariate regressions, but who view programming as a barrier rather than a primary skillset. In addition, CodeCaster can also be used those comfortable in their expertise but seek to become more productive.

2.2 Problem Justification

A traditional social science workflow involves receiving a massive data set or collecting data to create a set and then being tasked with finding underlying correlations. Traditionally, a student must manually scrub the data for errors, decide how to handle missing variables, and scour documentation to find the exact syntax for a regression model in Pandas or R. This process can be cumbersome. At best, it wastes time; at worst, it is offputting to those with great analytical skills but struggle with technology.

2.3 Competitors

From my research, the data analysis tooling space is heavily fragmented. Software like SPSS and Stata are powerful but feature a steep learning curve and expensive licensing fees. At the modern end, researchers attempt to use ChatGPT or Claude. However, generalized LLMs lack a dedicated execution environment that cater to analysts. CodeCaster differentiates itself by owning the entire pipeline: it ingests the data context, writes the code, executes it in a secure container, and interprets the output in a single, unified interface.

2.4 Monetization and Go-to-Market

The current version is designed as a lightweight, locally-hostable application to minimize financial overhead and for the sake of the project. A plausible go-to-market strategy involves a freemium model targeted at academic departments like at William & Mary or perhaps where specific online student communities can be found like Reddit. Basic dataset profiling and simple queries could be free, while executing complex models utilizing advanced Gemini API tiers could require a subscription or department-wide license. With initial feedback, I could further improve CodeCaster and make the experience even more distinct. Costs would only entail web hosting and Gemini API calls which would not be too costly in a hypothetical situation.

3 System Overview & Differentiation

CodeCaster is a single-page web application featuring a modern, glassmorphism-inspired UI built with Tailwind CSS. It replaces the traditional “split-screen” proposal with a more organized, multi-tabbed dashboard to prevent UI clutter. The user begins in the left-hand configuration panel. Upon uploading a CSV dataset, the platform immediately triggers the **Intelligent Codebook** feature. Unlike standard LLMs, CodeCaster reads a sample of the data and uses AI to classify every variable as *Nominal*, *Ordinal*, or *Continuous*. This Codebook is visually presented to the user and is secretly injected into all future code-generation prompts. This acts as a critical guardrail, ensuring the AI does not attempt to run a continuous model (like OLS regression) on nominal data, thus preserving statistical validity. Users select their preferred language (Python or R) and enter a plain-English prompt. The right-hand panel features four distinct tabs:

1. **Source Code:** Displays the optimized, commented script generated by the AI.
2. **Data Viewer:** A paginated, searchable data table for live data inspection.
3. **Analysis Results:** Displays the raw terminal output of the executed script, any generated statistical plots, and a plain-English AI interpretation of the p-values and correlations.
4. **Converse:** A contextual chat interface allowing the user to ask follow-up questions about their specific results.

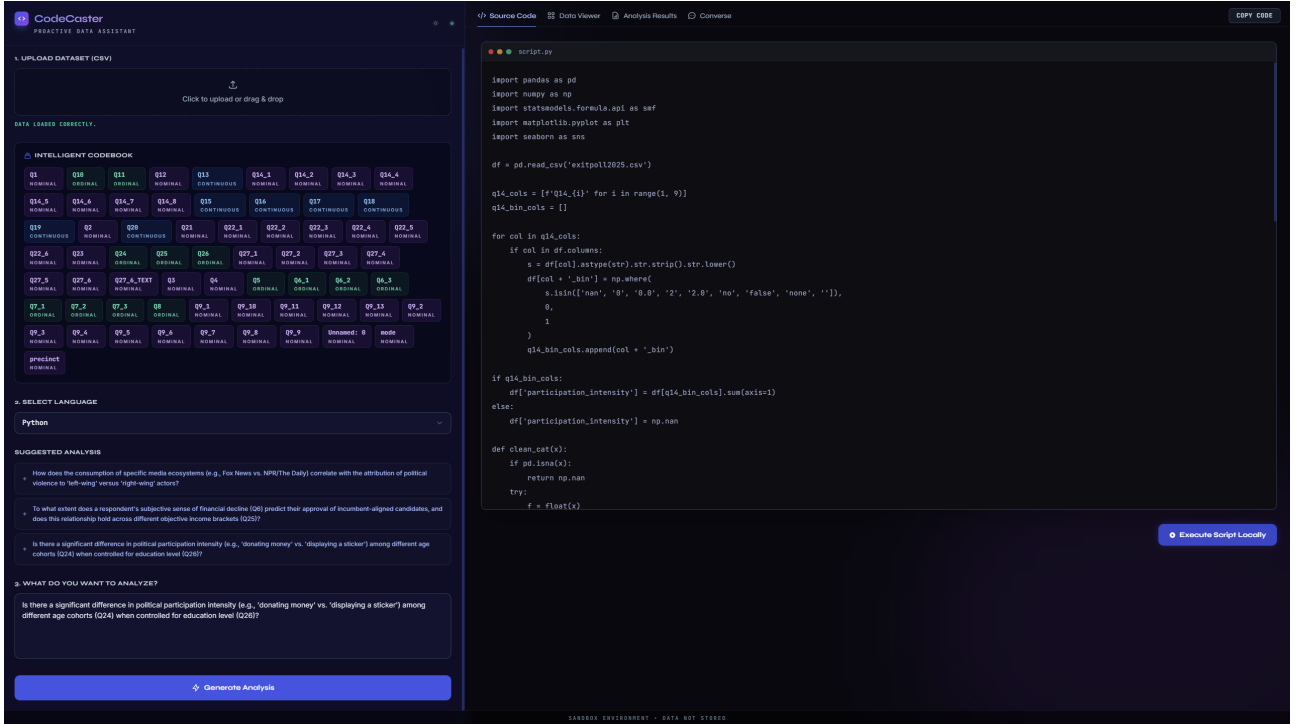


Figure 1: CodeCaster main interface showing the Intelligent Codebook classifying dataset variables and the generated Python script in the Source Code tab.

4 Technical Implementation

The application architecture relies on a robust separation of concerns, utilizing Vanilla JavaScript for the frontend and Python/Flask for the backend, entirely containerized via Docker.

4.1 Backend and Execution Environment

Security and environment consistency were paramount. The backend is a Flask server (`app.py`) that handles file uploads (capped at 16MB) and API routing. Because the platform executes AI-generated code, running this natively on a host machine poses security risks. Therefore, the entire application is bundled in a Docker container (`Dockerfile` and `docker-compose.yml`) using a `python:3.10-slim` base image, pre-loaded with necessary Python libraries (Pandas, Statsmodels) and R dependencies (`dplyr`, `ggplot2`). Code execution is sandboxed using Python’s `subprocess.run` with strict timeout parameters.

4.2 Multi-Model AI Architecture

CodeCaster does not rely on a single LLM call; it utilizes a highly sophisticated, multi-agent pipeline via the Google GenAI SDK to balance speed, cost, and reasoning capability:

- **Gemini 3.1 Pro Preview:** Used for the heavy lifting. It drafts the complex Python/R scripts based on the Codebook and performs the deep multimodal interpretation of the final terminal output and plots.
- **Gemini 3 Flash Preview:** Used as a rigorous validation layer. It acts as a “senior code reviewer,” checking the drafted code for syntax errors and missing imports before it is presented to the user. It is also used as a rapid formatting engine to clean up Markdown text.

- **Gemini 3.1 Flash Lite Preview:** Used for safety moderation (blocking malicious or off-topic prompts instantly) and for handling the conversational follow-up tab, ensuring rapid response times for chat.

4.3 Deviations from Proposal

The final product aligns closely with the mid-semester proposal, with two notable improvements. First, the proposed “split-screen” UI was upgraded to a tabbed interface to accommodate the breadth of features (Data Viewer, Terminal, Chat) without overwhelming the user. Second, the backend logic evolved from a simple prompt-to-code mechanism to a complex validation pipeline, adding the Intelligent Codebook to proactively enforce statistical rules.

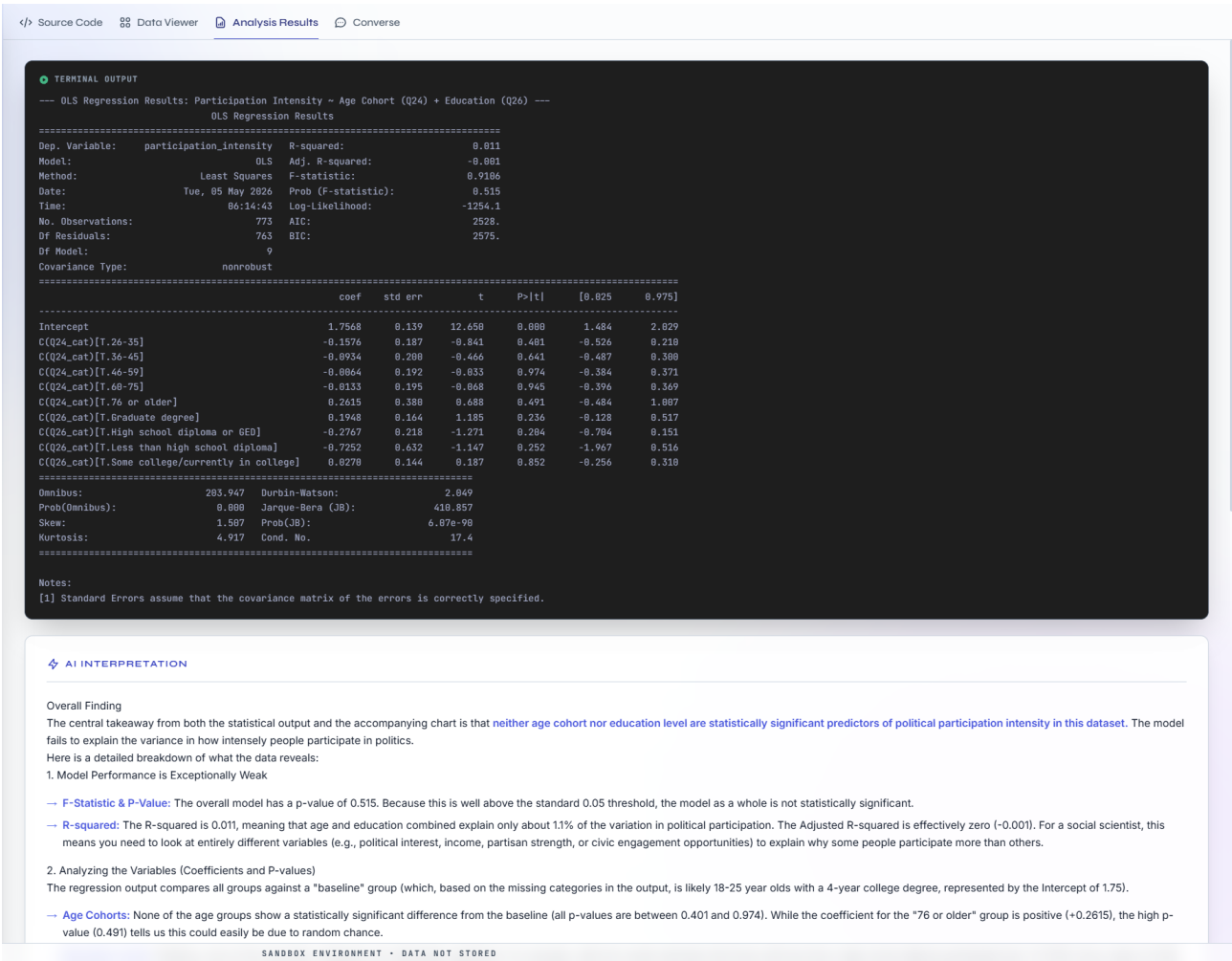


Figure 2: The top section of the Analysis Results tab, displaying the raw terminal execution output for the OLS regression and the beginning of the plain-English statistical AI interpretation.



Figure 3: The continuation of the Analysis Results tab, featuring the detailed diagnostic interpretation and the rendered bar chart visualization.

5 Vibe Coding Process & AI Reflection

Building this full-stack application solo within a single semester required leaning heavily on generative AI tools.

5.1 Workflow

My primary workflow relied on using Gemini’s web interface to vibe code. To maintain optimal performance, I proactively started new chat sessions whenever I felt the context window was running thin. I kept my prompts simple and straight to the point, only providing additional information when absolutely necessary or when the LLM was visibly struggling with a task. When I noticed the model making unauthorized deviations, such as reverting my customized UI to a messy, bland black-and-white design without prompting, I established explicit “DOs” and “DON’Ts” as guardrails to enforce consistency. Outside of the chat interface, I jotted down my ideas and project plans in separate documents, leveraging the LLM as an on-demand consultant to help set up any architecture or logic I was unfamiliar with.

5.2 Where Vibe Coding Excelled

The AI excelled at scaffolding boilerplate code and handling CSS styling. Creating the ambient animated background blobs, the glassmorphism panels, and the responsive flexbox layouts in Tailwind CSS took minutes instead of hours. Furthermore, the AI was exceptional in the early stages when there was not much code to work with and was able to create a foundation quickly.

5.3 Where Vibe Coding Struggled

The vibe coding process started to show signs of degradation when dealing with complex, interconnected state management and runtime nuances.

1. **Code Extraction Regex:** The AI initially struggled to write a robust regular expression to extract the raw code from the LLM’s Markdown fences (the “python blocks”). It required several manual iterations to handle edge cases where the LLM included conversational text alongside the code.
2. **Subprocess Context:** When implementing the local code execution via `subprocess.run`, the AI failed to account for the directory context of the generated plots. The script would run, but the saved `plot.png` would be lost in the container’s root directory rather than the designated temporary upload folder. This required human debugging and explicit pathway mapping.
3. **Vanilla JS DOM Manipulation:** Because I opted for Vanilla JS over a framework like React, keeping the DOM in sync across four different tabs (updating the code block, hiding the loading spinner, managing chat history) resulted in several race conditions that the AI could not logically trace without extensive human guidance.

The problem of scalability is one faced by many vibe-coded projects after a certain point. LLMs have become extremely competent in just a few years, but they will always have their limitations. The project becomes so large that the LLM fails to create meaningful additions or fix issues effectively, and because the developers don’t have much knowledge of the codebase, they hit a wall.

5.4 Takeaways

The most valuable lesson learned is that AI is a phenomenal executor but a poor architect. Open-ended prompts like “build a data analysis platform” result in messy, disjointed codebases. However, highly scoped prompts (e.g., “Write a Flask route that accepts a base64 image and a text string, and feeds it to Gemini 3.1 Pro for multimodal interpretation”) yield production-ready code. An AI’s ability is

only as good as the ability of the user to properly direct and manage it. Vibe coding CodeCaster felt like I was a supervisor who instructed my an AI model to complete tasks and then build upon those tasks with new tasks until I eventually felt satisfied. The importance of AI and Prompt Engineering in CS and the ability to use these skills to your advantage is already creating a gap and will continue to do so. Though, it is also important to note that being familiar with your codebase is crucial and should not be neglected despite these new tools.

6 Discussion and Limitations

While CodeCaster successfully achieves its goal, several limitations exist. First, the 16MB file upload limit imposed by the Flask configuration restricts the analysis of massive, enterprise-level datasets. Second, while Docker provides a layer of isolation, using `subprocess.run` to execute AI-generated code still carries inherent security risks if deployed to a public, internet-facing server without further hypervisor-level sandboxing. Finally, the quality of the generated statistical scripts is entirely bounded by the capabilities of the Gemini model; highly obscure statistical packages in R may occasionally result in hallucinations.

7 Future Work and Conclusion

Concrete next steps for CodeCaster involve evolving it from a reactive sandbox into a fully proactive research suite. CodeCaster proves that the barrier to entry for complex data analysis can be drastically lowered using Generative AI, serving as a testament to the viability of vibe coding for this course. By acting as the architect and utilizing AI for rapid implementation, a solo developer can build a secure, multi-model, containerized application that solves a genuine academic pain point. Future iterations will expand the platform's capabilities with several notable additions:

- **Proactive Analytics & Expanded Formats:** Automated statistical tests that suggest deep-divide analyses the moment a dataset is uploaded, alongside support for non-CSV file types and PDF codebook ingestion.
- **Advanced Data Wrangling:** Interactive UI tools to filter, recode, and drop columns, tracked seamlessly via an undo/redo version history sidebar.
- **Automated Reporting & Export:** Generation of editable, narrative-driven data reports tailored to specific writing styles, with the ability to export the entire workspace (data, scripts, plots, and report) as a zipped project file.
- **User Authentication & Session State:** A robust user account system featuring persistent chat histories, password-protected workspaces, and infrastructure scaling to support concurrent users.

Repository & Website: Please refer to the Git repository for more information on CodeCaster. In addition, you can test out CodeCaster [here](#).